

1. Define Machine Learning and list three domains where ML applied?

Machine Learning is a branch of artificial intelligence where computers learn patterns from data to make predictions or decisions without being explicitly programmed.

- Healthcare: ML analyses medical images for diagnosis and predicts patient outcomes.
- Finance: It detects fraud in transactions and forecasts market trends.
- Transportation: ML supports self-driving cars and traffic optimization.

2. What is inductive learning with an example?

Inductive learning is a machine learning approach where models generalize rules or patterns from specific training examples to predict outcomes on unseen data.

For example, a spam filter trained on labelled emails learns to classify new ones as spam or not by identifying common traits like keywords or sender patterns.

3. What is perceptron with an example?

A perceptron is the simplest artificial neural network unit, designed for binary classification by computing a weighted sum of inputs, adding a bias, and applying a step function to output 0 or 1.

For example, it can classify irises as Setosa or not by learning weights from petal length and width measurements during training

4. Define linear regression and state its primary assumption.

Linear regression is a supervised machine learning algorithm that models the linear relationship between a dependent variable and one or more independent variables by fitting a straight line to the data.

Its primary assumption is linearity, meaning the dependent variable changes proportionally with the independent variables

5. What is concept learning task?

Concept learning task is a machine learning problem where the goal is to infer a general Boolean function (target concept) from specific positive and negative training examples to accurately classify new instances

6. What is Overfitting?

Overfitting in machine learning occurs when a model learns the training data too closely, including noise and outliers, resulting in poor performance on unseen data

7. Define linear discriminants?

Linear Discriminant Analysis (LDA) also known as Normal Discriminant Analysis is supervised classification problem that helps separate two or more classes by converting higher-dimensional data space into a lower-dimensional space. It is used to identify a linear combination of features that best separates classes within a dataset.

8. What is Linear Separability?

In Machine Learning, a dataset is linearly separable if you can separate data points of different classes using a straight line (in 2D), a plane (in 3D), or more generally, a hyperplane (in higher dimensions).

9. What is interpolation?

Interpolation in machine learning is a technique to estimate unknown values within the range of a known dataset by constructing a function that passes through observed data points.

10. What is sigmoid activation function?

The sigmoid activation function in machine learning is a nonlinear S-shaped function defined as $\sigma(x) = 1 / (1 + e^{-x})$, mapping any real-valued input to a value between 0 and 1.

11. What is Tanh activation function?

The Tanh (hyperbolic tangent) activation function is a nonlinear S-shaped function defined as $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$, mapping input values to outputs between -1 and 1. Unlike sigmoid, it's zero-centred, aiding faster convergence in hidden layers of neural networks like RNNs.

12. What is difference between validation and testing datasets?

Validation datasets are used during model training to tune hyperparameters and prevent overfitting by evaluating performance iteratively, while testing datasets provide an unbiased final assessment of the model's generalization on completely unseen data after all tuning is complete.

13. What is the curse of dimensionality?

The curse of dimensionality in machine learning refers to the phenomenon where high-dimensional data spaces cause data points to become sparse, exponentially increasing the data volume needed for effective model training and degrading algorithm performance.

14. Define correlation and covariance?

Covariance measures how two variables change together, indicating the direction of their linear relationship—positive if they increase together, negative if one increases as the other decreases, and its value depends on the variables' scales.

Correlation, a standardized version of covariance (ranging from -1 to +1), quantifies both the strength and direction of the linear relationship, making it scale-independent and easier to interpret.

15. What are splines?

Splines in machine learning are piecewise polynomial functions used in regression to flexibly model nonlinear relationships by dividing the input space into segments at specified knots and fitting smooth polynomials within each. They help avoid overfitting compared to high-degree global polynomials while capturing complex data patterns.

16. What is convergence in backpropagation algorithm?

Convergence in the backpropagation algorithm refers to the point where the neural network's error (loss) stabilizes or reaches a minimum through iterative weight updates via gradient descent, indicating the model has learned optimal parameters. It typically occurs when changes in loss become negligible across epochs, though it may settle at local minima rather than the global minimum.

1. Explain the types of Machine Learning with examples

Machine learning is a subset of AI, which enables the machine to automatically learn from data, improve performance from past experiences, and make predictions.

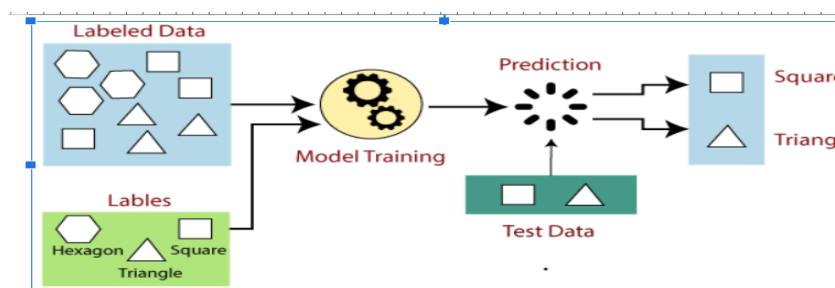
Machine learning contains a set of algorithms that work on a huge amount of data. These ML algorithms help to solve different business problems like Regression, Classification, Forecasting, Clustering, and Associations, etc. Based on the methods and way of learning, machine learning is divided into mainly four types, which are:

1. Supervised Machine Learning
2. Unsupervised Machine Learning
3. Semi-Supervised Machine Learning
4. Reinforcement Learning

Supervised Machine Learning :-

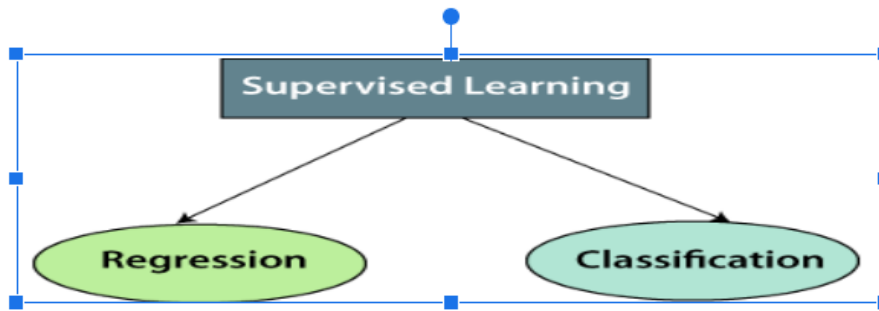
As its name suggests, Supervised machine learning is based on supervision. It means in the supervised learning technique, we train the machines using the "labelled" dataset, and based on the training, the machine predicts the output.

Here, the labelled data specifies that some of the inputs are already mapped to the output.



The main goal of the supervised learning technique is to map the input variable(x) with the output variable(y). Some real-world applications of supervised learning are Risk Assessment, Fraud Detection, Spam filtering, etc.

- o Classification
- o Regression



a. Classification –

Classification algorithms are used to solve the classification problems in which the output variable is categorical, such as "Yes" or No, Male or Female, Red or Blue, etc.

- The classification algorithms predict the categories present in the dataset. Some real-world examples of classification algorithms are Spam Detection, Email filtering, etc.

Some popular classification algorithms are given below:

o Random Forest Algorithm

o Decision Tree Algorithm

b) Regression

Regression algorithms are used to solve regression problems in which there is a linear relationship between input and output variables. These are used to predict continuous output variables, such as market trends, weather prediction, etc.

Some popular Regression algorithms are given below:

o Simple Linear Regression Algorithm

o Multivariate Regression Algorithm

o Decision Tree Algorithm

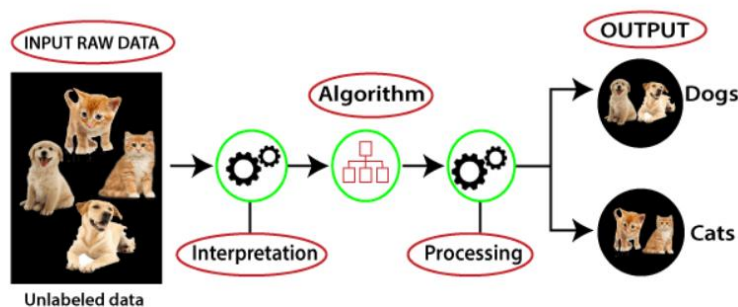
Unsupervised Machine Learning

Unsupervised learning is different from the Supervised learning technique; as its name suggests, there is no need for supervision.

It means, in unsupervised machine learning, the machine is trained using the unlabelled dataset, and the machine predicts the output without any supervision

The main aim of the unsupervised learning algorithm is to group or categories.

the unsorted dataset according to the similarities, patterns, and differences. Machines are instructed to find the hidden patterns from the input dataset



Categories of Unsupervised Machine Learning

Unsupervised Learning can be further classified into two types, which are given below:

- o Clustering
- o Association

1) Clustering

The clustering technique is used when we want to find the inherent groups from the data. It is a way to group the objects into a cluster such that the objects with the most similarities remain in one group and have fewer or no similarities with the objects of other groups.

An example of the clustering algorithm is grouping the customers by their purchasing behaviour.

Some of the popular clustering algorithms are given below:

- o K-Means Clustering algorithm
- o Mean-shift algorithm
- o DBSCAN Algorithm

2) Association

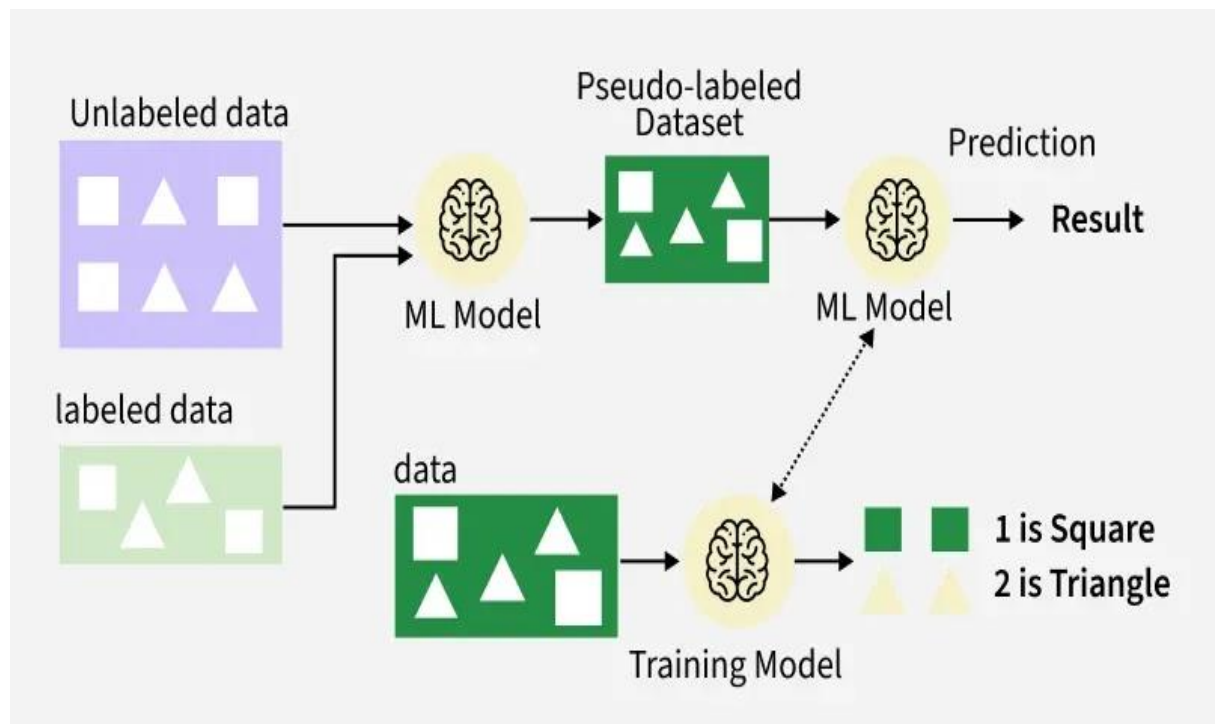
Association rule learning is an unsupervised learning technique, which finds interesting relations among variables within a large dataset.

The main aim of this learning algorithm is to find the dependency of one data item on another data item and map those variables accordingly so that it can generate maximum profit.

This algorithm is mainly applied in Market Basket analysis, Web usage mining, continuous production, etc.

Some popular algorithms of Association rule learning are

- Apriori Algorithm,
- Eclat,
- FP-growth algorithm.
- **Semi-Supervised Learning**
- Semi-Supervised learning is a type of Machine Learning algorithm that lies between Supervised and Unsupervised machine learning.
- It represents the intermediate ground between Supervised (With Labelled training data) and Unsupervised learning (with no labelled training data) algorithms and uses the combination of labelled and unlabelled datasets during the training period



Supervised learning is where a student is under the supervision of an instructor at home and college. Further, if that student is self-analysing the same concept without any help from the instructor, it comes under unsupervised learning. Under semi-supervised learning, the student has to revise himself after analyzing the same concept under the guidance of an instructor at college.

Advantages:

It is simple and easy to understand the algorithm.

It is highly efficient.

It is used to solve drawbacks of Supervised and Unsupervised Learning algorithms.

Disadvantages:

Iterations results may not be stable.

We cannot apply these algorithms to network-level data.

Reinforcement Learning

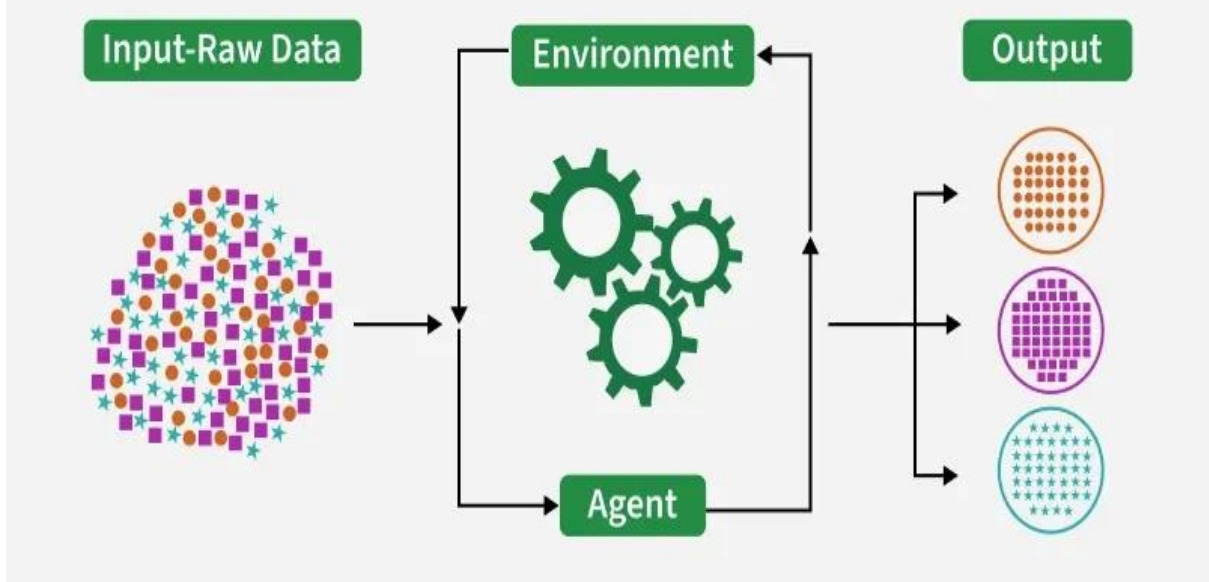
Reinforcement learning works on a feedback-based process, in which an AI agent (A software component) automatically explore its surrounding by hitting & trail, taking action, learning from experiences, and improving its performance.

Agent gets rewarded for each good action and get punished for each bad action; hence the goal of reinforcement learning agent is to maximize the rewards.

In reinforcement learning, there is no labelled data like supervised learning, and agents learn from their experiences only.

- **Agent:** The decision-maker that performs actions.
 - **Environment:** The world or system in which the agent operates.
 - **State:** The situation or condition the agent is currently in.
 - **Action:** The possible moves or decisions the agent can make.
- Reward:** The feedback or result from the environment based on the agent's action

Reinforcement Learning in ML



Reinforcement learning is categorized mainly into two types of methods/algorithms:

o Positive Reinforcement Learning: Positive reinforcement learning specifies

increasing the tendency that the required behaviour would occur again by adding something. It enhances the strength of the behaviour of the agent and positively impacts it.

o Negative Reinforcement Learning: Negative reinforcement learning works exactly opposite to the positive RL. It increases the tendency that the specific behaviour would occur again by avoiding the negative condition.

2. With suitable illustrations Explain Candidate Elimination algorithm

Smart way to compute version space using boundary sets (S and G)

Initialize G to the set of maximally general hypotheses in H

Initialize S to the set of maximally specific hypotheses in H

For each training example d , do

- If d is a positive example
 - Remove from G any hypothesis inconsistent with d
 - For each hypothesis s in S that is not consistent with d
 - Remove s from S
 - Add to S all minimal generalizations h of s such that
 - h is consistent with d , and some member of G is more general than h
 - Remove from S any hypothesis that is more general than another hypothesis in S
- If d is a negative example
 - Remove from S any hypothesis inconsistent with d
 - For each hypothesis g in G that is not consistent with d
 - Remove g from G
 - Add to G all minimal specializations h of g such that
 - h is consistent with d , and some member of S is more specific than h
 - Remove from G any hypothesis that is less general than another hypothesis in G

Sky	Temperature	Humid	Wind	Water	Forest	Output
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	change	yes

S0: ($\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset$) Most Specific Boundary

G0: (?, ?, ?, ?, ?, ?) Most Generic Boundary

The first example is positive, the hypothesis at the specific boundary is inconsistent, hence we extend the specific boundary, and the hypothesis at the generic boundary is consistent hence we retain it

S1: (Sunny, Warm, Normal, Strong, Warm, Same)

G1: (?, ?, ?, ?, ?, ?)

The second example is positive, again the hypothesis at the specific boundary is inconsistent, hence we extend the specific boundary, and the hypothesis at the generic boundary is consistent hence we retain it.

S2: (Sunny, Warm, ?, Strong, Warm, Same)

G2: (?, ?, ?, ?, ?, ?)

The third example is negative, the hypothesis at the specific boundary is consistent, hence we retain it, and hypothesis at the generic boundary is inconsistent hence we write all consistent hypotheses by removing one "?" (question mark) at a time.

S3: (Sunny, Warm, ?, Strong, Warm, Same)

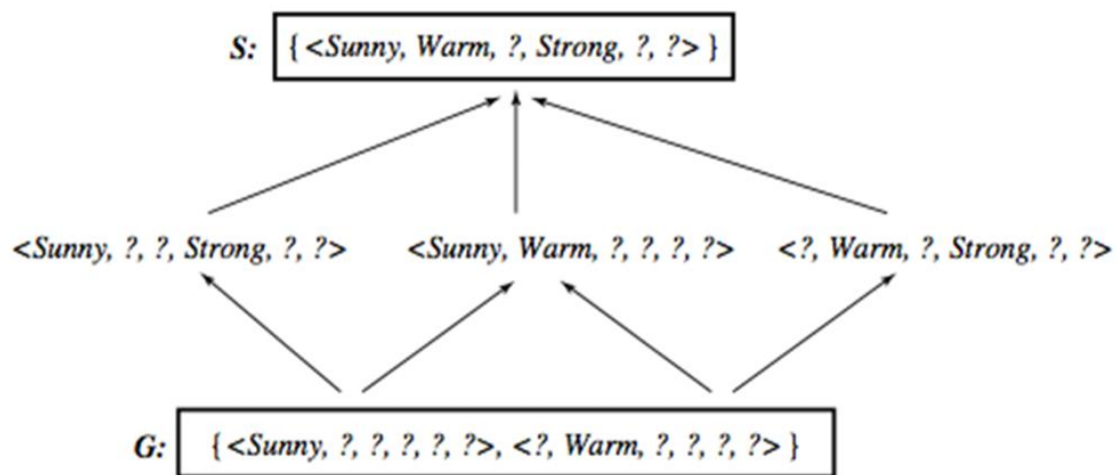
G3: (Sunny, ?, ?, ?, ?, ?) (? , Warm, ?, ?, ?, ?) (? , ?, ?, ?, ?, Same)

The fourth example is positive, the hypothesis at the specific boundary is inconsistent, hence we extend the specific boundary, and the consistent hypothesis at the generic boundary are retained.

S4: (Sunny, Warm, ?, Strong, ?, ?)

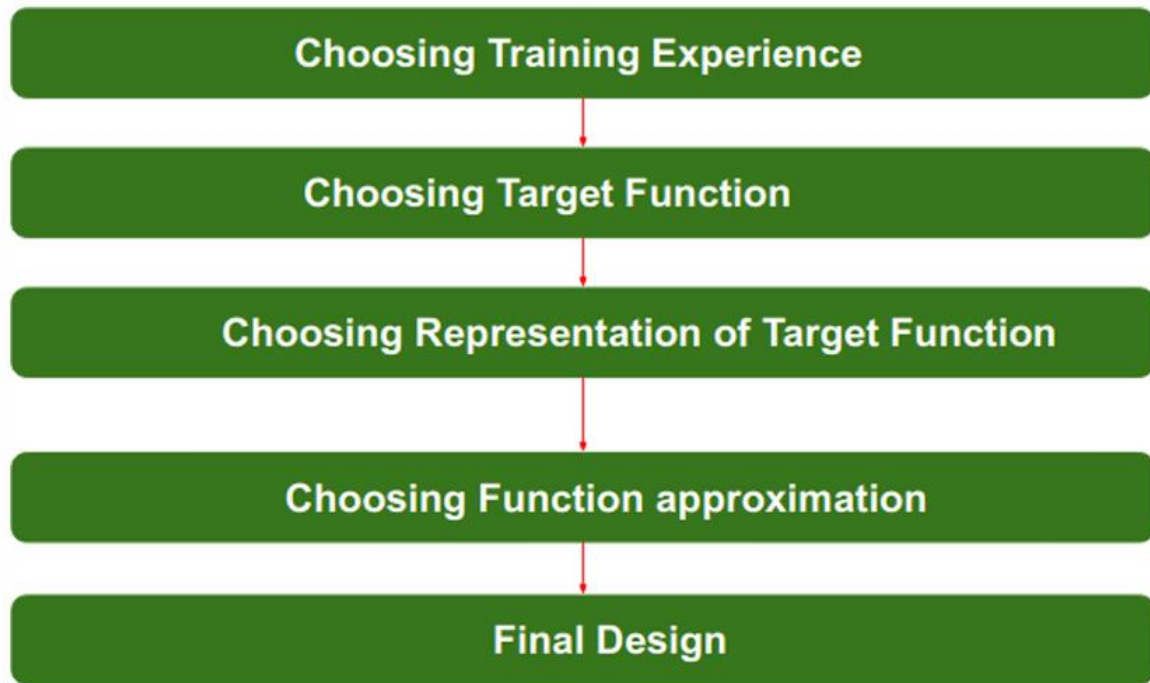
G4: (Sunny, ?, ?, ?, ?, ?) (? , Warm, ?, ?, ?, ?).

Learned Version Space by Candidate Elimination Algorithm for given data set is:



3. Explain How to a design learning system?

. In order to get perfect and efficient system, we need to follow a particular design. For proper design, we must follow several steps as shown below



Choosing the Training Experience: The first design choice we face is to choose the type of training experience from which our system will learn.

The type of training experience available can have a significant impact on success or failure of the learner. One key attribute is whether the training experience provides direct or indirect feedback regarding the choices made by the performance system.

For example, in learning to play checkers, the system might learn from direct training examples consisting of individual checkers board states and the correct move for each. In order to complete the design of the learning system, we must now choose

1. the exact type of knowledge to be learned
2. a representation for this target knowledge
3. a learning mechanism

Choosing the Target Function: The next design choice is to determine exactly what type of knowledge will be learned and how this will be used by the performance program. Let us begin with a checkers-playing program that can generate the legal moves from any board state. The program needs only to learn how to choose the best move from among these legal moves.

Choosing a Representation for the Target Function:

X1: the number of black pieces on the board

x2: the number of red pieces on the board

x3: the number of black kings on the board

x4: the number of red kings on the board

x5: the number of black pieces threatened by red (i.e., which can be captured on red's next turn)

X6: the number of red pieces threatened by black

Thus, our learning program will represent $V(b)$ as a linear function of the form

$$V(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

where w_0 through w_6 are numerical coefficients, or weights, to be chosen by the learning algorithm.

Choosing a Function Approximation Algorithm

In order to learn the target function f we require a set of training examples, each describing a specific board state b and the training value $V_{train}(b)$ for b .

ESTIMATING TRAINING VALUES

Rule for estimating training values.

$$V_{train}(b) \approx V(\text{Successor}(b))$$

ADJUSTING THE WEIGHTS

$$E \equiv \sum_{\langle b, V_{train}(b) \rangle \in \text{training examples}} (V_{train}(b) - \hat{V}(b))^2$$

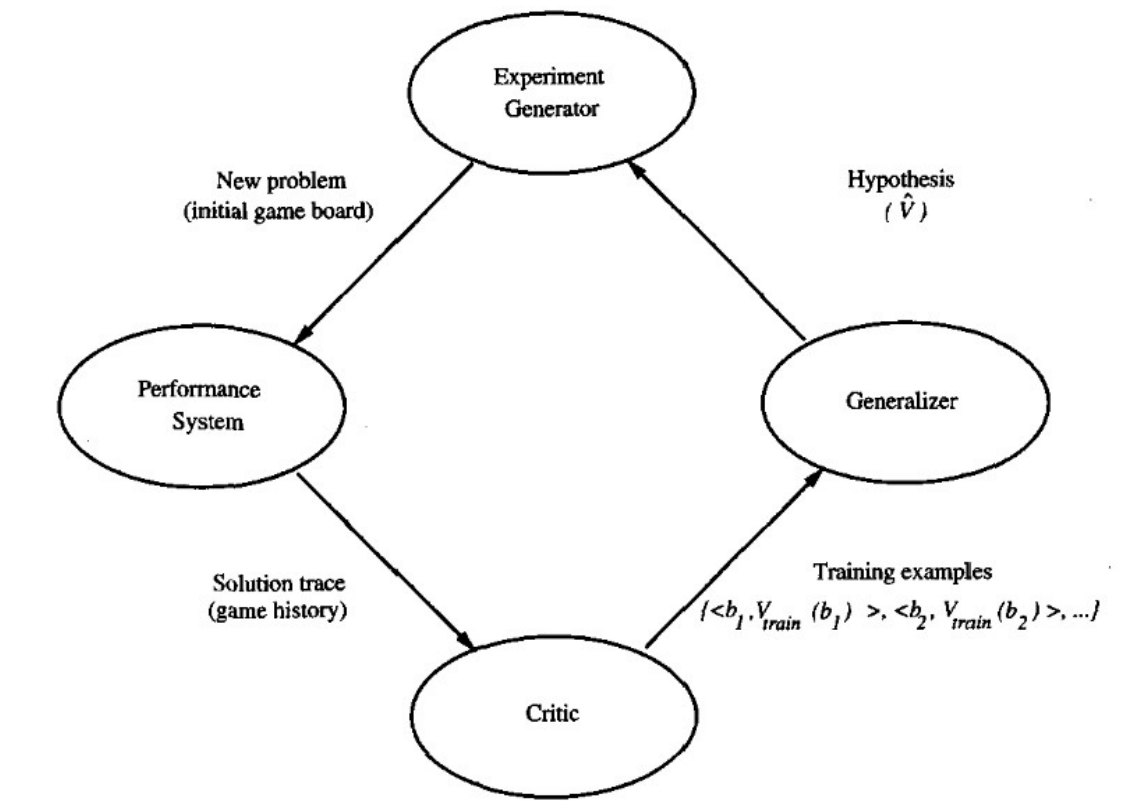
The Final Design:

The Performance System is the module that must solve the given performance task, in this case playing checkers, by using the learned target function(s). It takes an instance of a new problem (new game) as input and produces a trace of its solution (game history) as output.

The **Critic** takes as input the history or trace of the game and produces as output a set of training examples of the target function. As shown in the diagram, each training example in this case corresponds to some game state in the trace, along with an estimate V_{train} , of the target function value for this example.

The **Generalizer** takes as input the training examples and produces an output hypothesis that is its estimate of the target function. It generalizes from the specific training examples, hypothesizing a general function that covers these examples and other cases beyond the training examples.

The **Experiment Generator** takes as input the current hypothesis (currently learned function) and outputs a new problem (i.e., initial board state) for the Performance System to explore. Its role is to pick new practice problems that will maximize the learning rate of the overall system



4. Illustrate the role of hypothesis space and concept learning in ML?

Concept learning is the process of inferring a general rule from a set of observed examples to distinguish between positive and negative instances of that concept.

This process of identifying the right concept can be effectively framed as a search problem, a perspective known as Concept Learning as Search.

Concept Learning as a Search Problem

- Concept learning can be viewed as searching through a hypothesis space (all possible rules).
- The goal is to find a hypothesis that is consistent with all given training examples (both positive and negative).
- The search is typically organized from general to specific hypotheses.
- This structure helps in efficient exploration of the hypothesis space.

Concept learning requires the following:

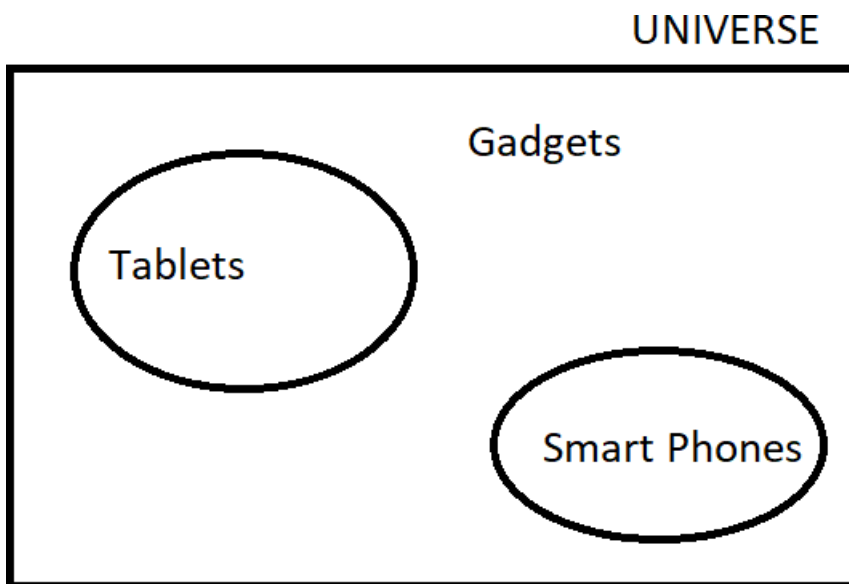
- Training Data
- Target Concept
- Actual Data Objects

General Hypothesis

- Hypothesis, in general, is an explanation for something. The general hypothesis basically states the general relationship between the major variables. For example, a general hypothesis for ordering food would be I want a burger.
- $G = \{ '?', '?', '?', \dots, '?' \}$

Specific Hypothesis

- The specific hypothesis fills in all the important details about the variables given in the general hypothesis. The more specific details into the example given above would be I want a cheeseburger with a chicken pepperoni filling with a lot of lettuce.
- $S = \{\Phi', \Phi', \Phi', \dots, \Phi'\}$
- Finding all the **consistent hypothesis** or the **concepts**.
- Size: Large, Small $\rightarrow X1$
- Color: Black, Blue $\rightarrow X2$
- **Screen Size: Flat, Folded $\rightarrow X3$**
- Shape: square, rect $\rightarrow X4$



General representation of the Concept = $\langle X1, X2, X3, X4 \rangle$

Individual Representation

Tablet = $\langle \text{Large, Black, Flat, Square} \rangle$

Smart Phone = $\langle \text{Small, Blue, Folded, Rectangle} \rangle$ Number of possible instances for a particular concept = 2^d .

Where 'd' is number of features.

Total Number of possible concepts = 2^d .

The Version Space $VS_{H,D}$ is the subset of hypothesis from H which are consistent with the training examples

$$D. VS_{H,D} = \{ h \in H \mid \text{consistent}(h,D) \}$$

- A version space is the set of all hypotheses that are consistent with the observed training examples.
- It includes all the hypotheses from the hypothesis space H that correctly classify both positive and negative examples seen so far.

Once all training examples are processed, the version space includes all hypotheses consistent with the data.

The learner can now:

- Predict unseen instances confidently if all hypotheses agree.
 - Ask queries to reduce the space (active learning).
 - Identify errors if the space becomes empty
-
- An hypothesis h is said be consistent hypothesis with a set of training examples D , iff $h(x)=c(x)$ for each example in D .

Example	Citations	Size	inLibrary	Price	Editions	Buy
1	Some	Small	No	Affordable	One	No
2	Many	Big	No	Expensive	Many	Yes

$h1 = \langle ?, ?, \text{No}, ?, \text{Many} \rangle$ -Consistent

$h2 = \langle ?, ?, \text{No}, ?, ? \rangle$ -Not Consistent

5. Describe the concept of RBF networks?

A radial basis function (RBF) neural network is a type of artificial neural network that uses radial basis functions as activation functions. It typically consists of three layers: an input layer, only one hidden layer, and an output layer. The hidden layer applies a radial basis function, usually a Gaussian function. RBF neural networks are highly versatile and are extensively used in pattern classification tasks, function approximation, and a variety of machine learning applications. They are especially known for their ability to handle non-linear problems effectively.

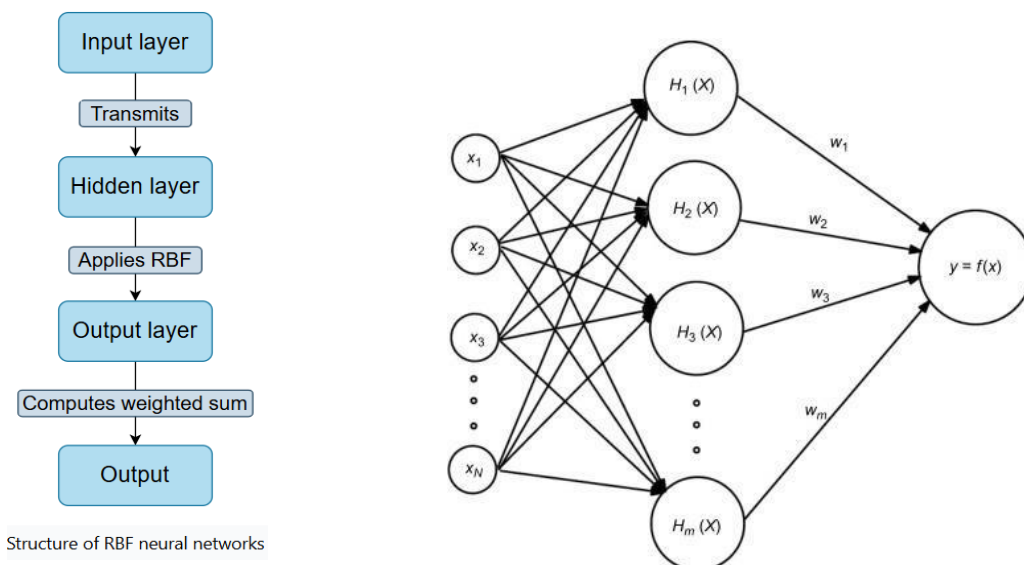
Structure of RBF neural networks

An RBF neural network typically comprises three layers:

- **Input layer:** This layer simply transmits the inputs to the neurons in the hidden layer.
- **Hidden layer:** Each neuron in this layer applies a radial basis function to the inputs it receives. RBF has strictly one hidden layer.
- **Output layer:** Each neuron in this layer computes a weighted sum of the outputs from the hidden layer, resulting in the final output.

Working of RBF

- When dealing with **non-linear data**, we aim to convert it into **linearly separable data**.
- To achieve this, every hidden layer neuron uses a **non-linear radial basis function** as the activation function, transforming the data into a **higher-dimensional space**.



Types of Radial Basis Functions:

1. **Gaussian RBF** (most common): x =Input c = Center r = Radius

$$H(x) = e^{-\frac{(x-c)^2}{r^2}}$$

2. **Multiquadric RBF:**

$$H(x) = \sqrt{x^2 + (x - c)^2}$$

Algorithm of RBF

Input & Output

- Input: A set of **input vectors** ($x_1, x_2, \dots, x_n$)
- Output: **y_n**

Step 1: Initialize Weights

- Assign weights for each connection from **hidden layer** to **output layer**.
- Initially, weights are randomly assigned in the range **[-1,1]**.

Forward Phase

Step 1: Input Layer Computation

- Each node in the **input layer** directly passes its input:

$$I_i = x_i, \quad O_i = I_i$$

- Input at **Node i**: I_i
- Output at **Node i**: O_i (same as input)

Step 2: Hidden Layer Computation

- The hidden layer applies the **Radial Basis Function**:

$$H_j(x) = e^{-\frac{(x-c_j)^2}{r^2}}$$

- x = Input
 - c_j = Center
 - r = Radius
-
- The distance between **input x** and **center c** determines the activation.

Step 3: Output Layer Computation

- Compute **weighted sum** of hidden layer outputs:

$$F(x) = \sum W_{jk} H_j(x)$$

- W_{jk} = Weight from hidden neuron j to output neuron k
- $H_j(x)$ = Output of hidden neuron
- The final output y_o is obtained.
- If the output is **not satisfactory**, we perform **backpropagation**.

Backward Phase (Training)

1. Train the hidden layer using backpropagation.
2. Update weights **between** hidden layer **and** output layer.

Key Characteristics of RBFs

- **Radial Basis Functions:** These are real-valued functions dependent solely on the distance from a central point. The Gaussian function is the most commonly used type.
- **Dimensionality:** The network's dimensions correspond to the number of predictor variables.
- **Center and Radius:** Each RBF neuron has a center and a radius (spread). The radius affects how broadly each neuron influences the input space.

Advantages of RBF Networks

1. **Universal Approximation:** RBF Networks can approximate any continuous function with arbitrary accuracy given enough neurons.
2. **Faster Learning:** The training process is generally faster compared to other neural network architectures.
3. **Simple Architecture:** The straightforward, three-layer architecture makes RBF Networks easier to implement and understand.

Applications of RBF Networks

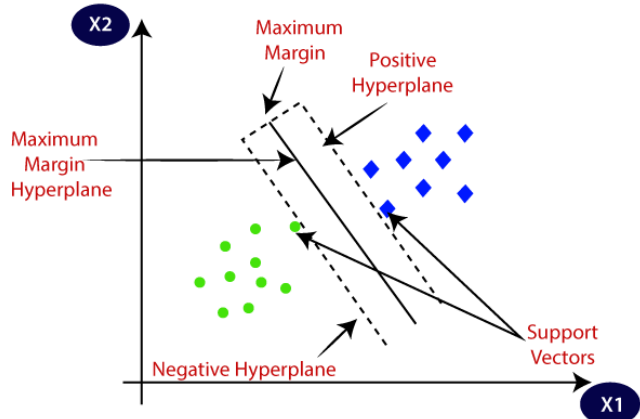
- **Classification:** RBF Networks are used in pattern recognition and classification tasks, such as speech recognition and image classification.
- **Regression:** These networks can model complex relationships in data for prediction tasks.
- **Function Approximation:** RBF Networks are effective in approximating non-linear functions.

6. Describe support vector machine algorithm with an example?

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:

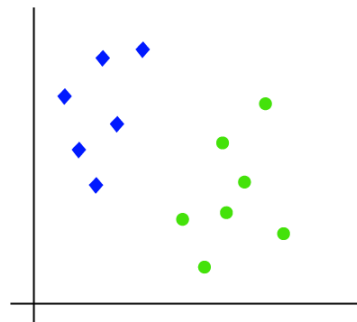


Types of SVM

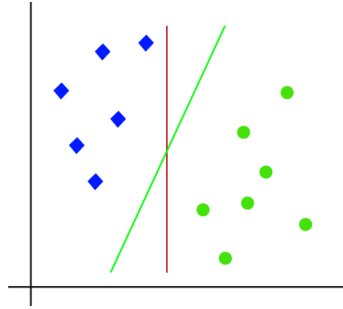
SVM can be of two types:

Linear SVM:

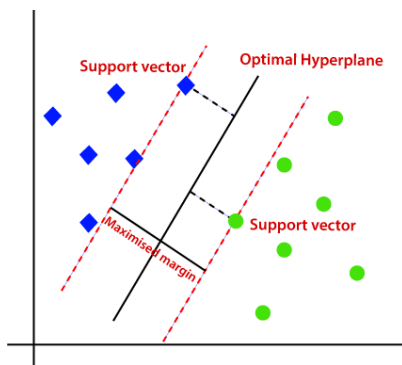
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

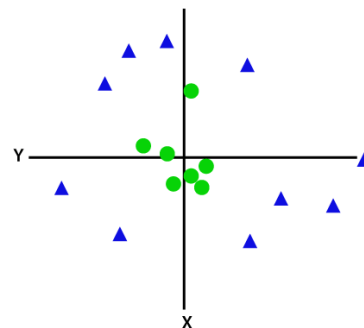


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin.



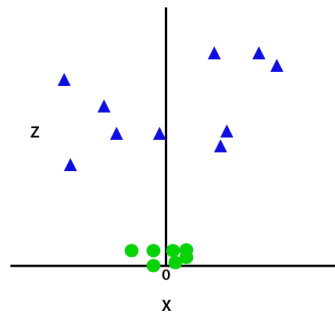
Kernel or Non-Linear SVM:

If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



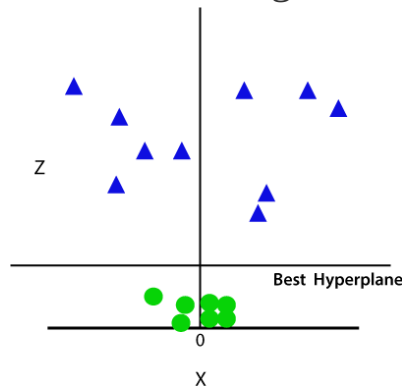
So, to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third-dimension z. It can be calculated as:

$$z = x^2 + y^2$$

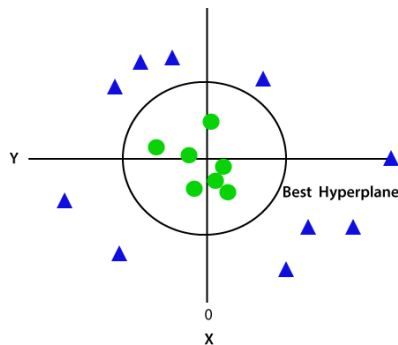


By adding the third dimension, the sample space will become as below image:

So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the



x-axis. If we convert it in 2d space with $z=1$, then it will become as:

Hence, we get a circumference of radius 1 in case of non-linear data

How does Support Vector Machine Algorithm Work?

1. **Plot the Data:** Each data point is represented in n -dimensional space (n = number of features). For example, if you have two features, you can plot the data on a 2D graph.

Each data point is represented as x_i in an n -dimensional space:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

Where:

- $x_i \in \mathbb{R}^n$ is the feature vector for the i -th data point.
- $y_i \in \{-1, +1\}$ is the label (class) of the data point.

For example:

- In 2D: $x_i = (x_{i1}, x_{i2})$.
- In 3D: $x_i = (x_{i1}, x_{i2}, x_{i3})$.

2. **Find the Hyperplane:** SVM finds the hyperplane (a straight line in 2D, a flat plane in 3D, or more generally, an n -dimensional plane) that separates the two classes of data points with the **maximum margin**.
 - **Maximum Margin:** This is the largest possible distance between the hyperplane and the nearest data points from both classes.
 - These closest points are called **support vectors** because they “support” the hyperplane.

The hyperplane is defined as:

$$w \cdot x + b = 0$$

Where:

- w : Normal vector of the hyperplane.
- b : Bias term (offset from the origin).

Maximum Margin:

The margin is the distance between the hyperplane and the closest data points (called **support vectors**).

The formula for the margin is:

$$\text{Margin} = \frac{2}{\|w\|}$$

We maximize this margin by solving the following optimization problem:

$$\min \frac{1}{2} \|w\|^2$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

3. **Separate the Classes:** The hyperplane divides the data into two regions, each representing one class. For example:
- One side of the line = Class A.
 - Other side = Class B.

The hyperplane divides the data into two regions:

- One side of the hyperplane: $w \cdot x + b > 0$ ($y = +1$, Class A).
- Other side: $w \cdot x + b < 0$ ($y = -1$, Class B).

For classification, the decision rule is:

$$f(x) = \text{sign}(w \cdot x + b)$$

4. **Non-Linearly Separable Data:** If the data cannot be separated with a straight line (e.g., spiral data), SVM uses something called a **kernel trick** to transform the data into a higher dimension where it becomes linearly separable.
- **Kernel Functions:** Mathematical functions like polynomial, RBF (Radial Basis Function), etc., are used to transform the data.

The kernel function $K(x_i, x_j)$ replaces $w \cdot x$, allowing calculations in the transformed space without explicitly transforming the data.

Common kernel functions:

1. **Linear Kernel:**

$$K(x_i, x_j) = x_i \cdot x_j$$

2. **Polynomial Kernel:**

$$K(x_i, x_j) = (x_i \cdot x_j + c)^d$$

3. **RBF (Gaussian) Kernel:**

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

The decision function with kernels becomes:

$$f(x) = \text{sign}\left(\sum_{i=1}^n \alpha_i y_i K(x_i, x) + b\right)$$

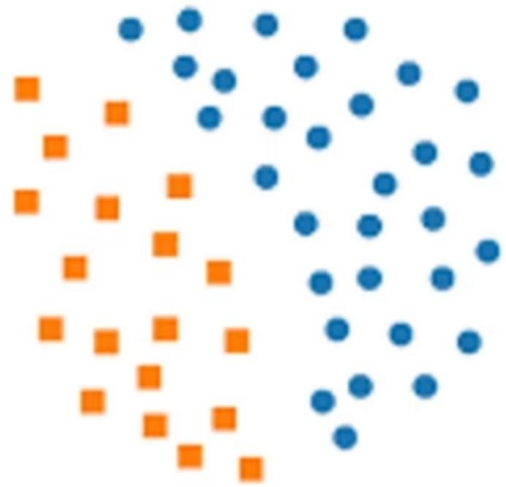
Where α_i are the weights learned from the training process.

7.Explain methods for handling imbalanced datasets with suitable examples?

Balanced Dataset: —

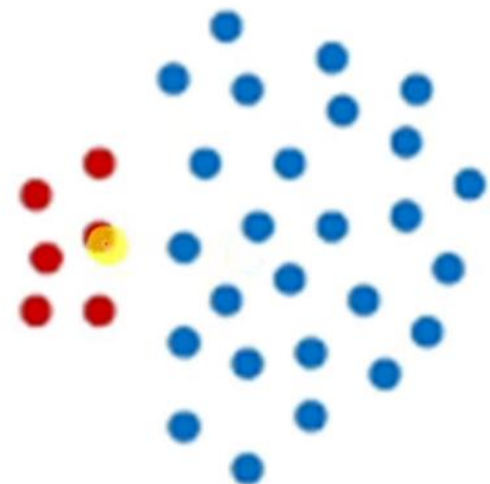
Let's take a simple example if in our data set we have positive values which are approximately same as negative values.

Then we can say our dataset in balance



• Imbalanced Dataset: —

- If there is the very high difference between the positive values and negative values.
- Then we can say our dataset in Imbalance Dataset.



Problem with an Imbalanced Datasets

- Standard Machine Learning techniques such as Decision Tree and Logistic Regression have a bias towards the **majority** class, and they tend to ignore the minority class.

- Let's say you are working on fraud detection.
- Here the fraudulent transaction is relatively rare;
- So you are training your model and get over 95% accuracy.
- When you give inputs to your model, your model is always predicting "Not a Fraud Transaction" every time.

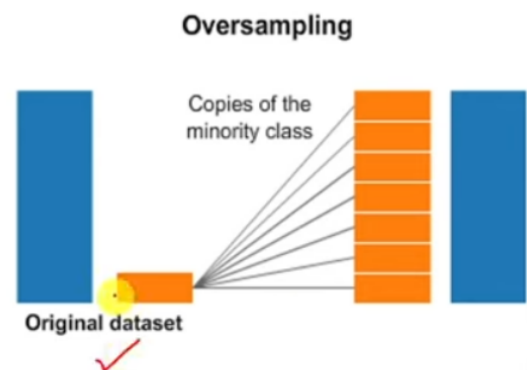
Techniques to Solve the Class Imbalance Problem

1. Use the right evaluation metrics:

- **Confusion Matrix:** a table showing correct predictions and types of incorrect predictions.
- **Precision:** the number of true positives divided by all positive predictions
- **Recall:** the number of true positives divided by the number of positive values in the test data.
- **F1 Score:** the weighted average of precision and recall.

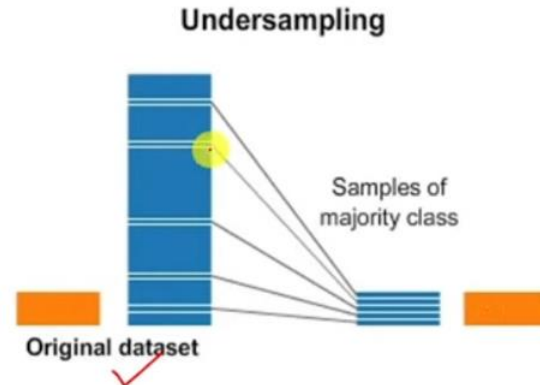
2. Over-sampling (Up Sampling):

- Over-sampling increases the number of minority class members in the training set.
- The advantage of over-sampling is that no information from the original training set is lost, as all observations from the minority and majority classes are kept.



3. Under-sampling (Down Sampling):

- Under-sampling, on contrary to over-sampling, aims to reduce the number of majority samples to balance the class distribution.
- Since it is removing observations from the original data set, it might discard useful information.

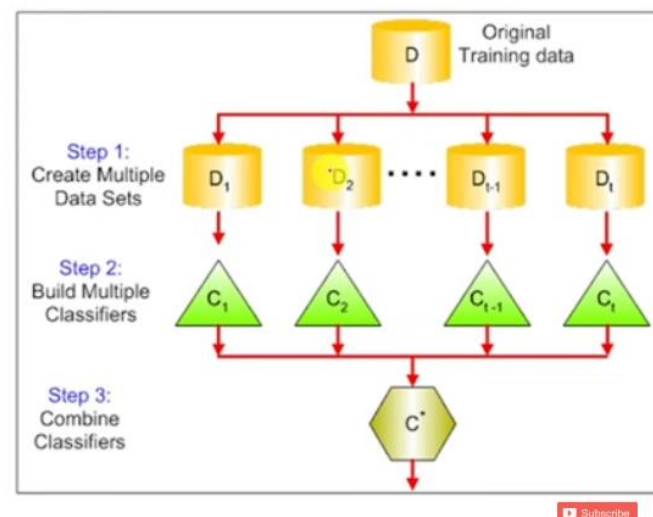


4. Feature selection:

- In order to tackle the imbalance problem, we calculate the one-sided metric such as correlation coefficient (CC) and odds ratios (OR) or two-sided metric evaluation such as information gain (IG) and chi-square (CHI) on both the positive class and negative class.
- Based on the scores, we then identify the significant features from each class and take the union of these features to obtain the final set of features.

5. Ensemble Learning Techniques:

- The ensemble-based method is another technique which is used to deal with imbalanced data sets
- It mainly combines the outputs of multiple base learners.
- There are various approaches in ensemble learning such as Bagging, Boosting, stacking etc.



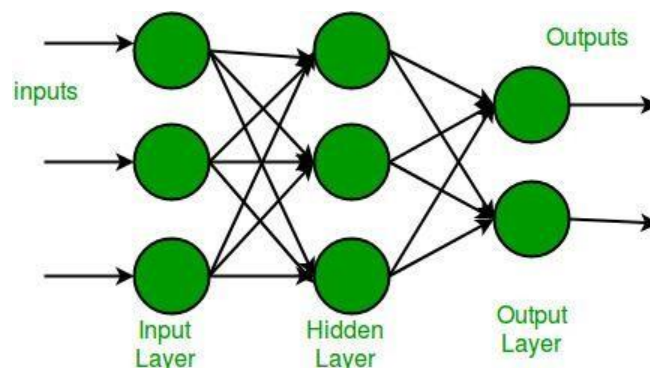
8.Explain & illustrate the structure of a Multi-Layer Perceptron (MLP) with a neat diagram?

MULTI-LAYER PERCEPTRON

A **Multi-Layer Perceptron (MLP)** consists of fully connected dense layers that transform input data from one dimension to another. It is called “multi-layer” because it contains an input layer, one or more hidden layers, and an output layer. The purpose of an MLP is to model complex relationships between inputs and outputs, making it a powerful tool for various machine learning tasks. MLP (Multi-Layer Perceptron) is primarily used for supervised learning, as it is a type of artificial neural network that requires labeled data to train and learn relationships between input features and target outputs, making it suitable for tasks like classification and regression.

The key components of Multi-Layer Perceptron include:

- **Input Layer:** Each neuron (or node) in this layer corresponds to an input feature. For instance, if you have three input features, the input layer will have three neurons.
- **Hidden Layers:** An MLP can have any number of hidden layers, with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer:** The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.



Every connection in the diagram is a representation of the fully connected nature of an MLP. This means that every node in one layer connects to every node in the next layer. As the data moves through the network, each layer transforms it until the final output is generated in the output layer.

Step 1: Forward Propagation

In **forward propagation**, the data flows from the input layer to the output layer, passing through any hidden layers. In forward propagation, the **MLP computes predictions**, regardless of whether we use **MSE** or **BCE**. The choice of loss function (MSE or BCE) depends on whether the task is **regression or classification**. Each neuron in the hidden layers processes the input as follows:

1. **Weighted Sum:** The neuron computes the weighted sum of the inputs:

- $z = \sum_i w_i x_i + b$

- Where:

- x_i is the input feature.
 - w_i is the corresponding weight.
 - b is the bias term.

2. **Activation Function:** An activation function is a mathematical function applied to the output of a neuron. It introduces non-linearity into the model, allowing the network to learn and represent complex patterns in the data. Without this non-linearity feature, a neural network would behave like a linear regression model, no matter how many layers it has.

The activation function decides whether a neuron should be activated by calculating the weighted sum of inputs and adding a bias term. This helps the model make complex decisions and predictions by introducing non-linearities to the output of each neuron. Neural networks consist of neurons that operate using **weights, biases, and activation functions**

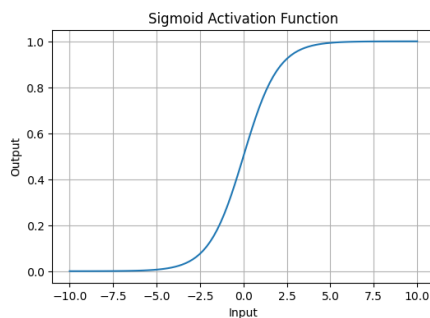
1. **Sigmoid:** Sigmoid function is a mathematical function that has an “S”-shaped curve (sigmoid curve). The sigmoid function is one of the most commonly used activation functions in Machine learning and Deep learning. It is particularly useful in neural networks, where it introduces non-linearity, allowing the model to handle complex patterns in the data.

Sigmoid function is also known as the squashing function, as it takes the input from the previously hidden layer and squeezes it between 0 and 1. So a value fed to the sigmoid function will always return a value between 0 and 1, no matter how big or small the value is fed.

The formula of the sigmoid activation function is:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Here, e is the base of the natural logarithm (approximately equal to 2.71828), and x is the input to the function.



Step 2: Loss Function

A **loss function** is a mathematical function that measures how well a model's predictions match the true outcomes. It provides a quantitative metric for the accuracy of the model's predictions, which can be used to guide the model's training process. The goal of a loss function is to guide optimization algorithms in adjusting model parameters to reduce this loss over time. Once the network generates an output, the next step is to calculate the **loss** using a loss function. In supervised learning, this compares the predicted output to the actual label.

For a classification problem, the commonly used binary cross-entropy loss function is:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

Where:

- y_i is the actual label.
- $\hat{y}_i$ is the predicted label.
- N is the number of samples.

For regression problems, the mean squared error (MSE) is often used:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Step 3: Backpropagation

The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through **backpropagation**. **Both MSE and BCE can be used in backpropagation**. Backpropagation **computes gradients of the chosen loss function** (MSE or BCE) and updates the network's weights using **gradient descent**.

1. **Gradient Calculation:** The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
2. **Error Propagation:** The error is propagated back through the network, layer by layer.
3. **Gradient Descent:** The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss

For both **regression (MSE loss)** and **classification (BCE loss)**, the weights are updated using the gradient descent formula:

$$W = W - \eta \frac{\partial L}{\partial W}$$
$$b = b - \eta \frac{\partial L}{\partial b}$$

where:

- η is the **learning rate** (controls the step size).
- $\frac{\partial L}{\partial W}$ is the **gradient of the loss function w.r.t. weights**.
- $\frac{\partial L}{\partial b}$ is the **gradient of the loss function w.r.t. bias**.

$$W = W - \eta \frac{\partial L}{\partial W}$$
$$b = b - \eta \frac{\partial L}{\partial b}$$

Step 4: Iteration

- Forward and backward propagation repeat over multiple epochs until the model converges (i.e., achieves an acceptable error rate).

9. Derive backpropagation of classification?

Backpropagation for Classification (BCE Loss)

We use **Binary Cross-Entropy (BCE) loss**, which is used for **binary classification** (e.g., Spam vs. Not Spam).

Step 1: Forward Propagation (Compute Prediction)

Each neuron performs:

$$z = WX + b$$
$$y_{\text{pred}} = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (\text{sigmoid activation})$$

where y_{pred} is the predicted probability.

Step 2: Compute Binary Cross-Entropy (BCE) Loss

$$L = -\frac{1}{N} \sum [y_{\text{true}} \log(y_{\text{pred}}) + (1 - y_{\text{true}}) \log(1 - y_{\text{pred}})]$$

Step 3: Compute Gradients (Find Errors)

1. Compute **error signal**:

$$\delta_{\text{out}} = y_{\text{pred}} - y_{\text{true}}$$

- If $y_{\text{true}} = 1$ and $y_{\text{pred}} = 0.1$, error is **high**.
- If $y_{\text{true}} = 1$ and $y_{\text{pred}} = 0.9$, error is **low**.

2. Compute **gradient w.r.t weights**:

$$\frac{\partial L}{\partial W} = \delta_{\text{out}} \cdot X$$

3. Compute **gradient w.r.t bias**:

$$\frac{\partial L}{\partial b} = \delta_{\text{out}}$$

Step 4: Update Weights

$$W = W - \eta \frac{\partial L}{\partial W}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

1. A maximally specific hypothesis means [C]
A . Considers all possible generalizations **B .** Ignores negative examples
C . Covers only observed positive examples **D .** Is the most general hypothesis

2. The process of calculating outputs from inputs in MLP is called [A]
A . Forward pass **B .** Backward pass **C .** Gradient descent **D .** Weight initialization

3. Reinforcement learning is based on [B]
A . Error correction **B .** Rewards and punishments **C .** Data clustering **D .** Linear regression

4. Which of the following cannot be learned by a perceptron? [C]
A . AND function **B .** OR function **C .** XOR function **D .** Linearly separable data

5. Generalization in ML refers to [D]
A . Remembering all training data **B .** Fitting only training data
C . Increasing dataset size **D .** Performing well on unseen data

6. The version space is defined as [B]
A . All possible hypotheses **B .** Set of hypotheses consistent with training examples
C . Hypotheses with minimum error **D .** General hypotheses only

7. The activation functions in hidden layers of MLP are usually [B]
A . Linear **B .** Non-linear **C .** Step function only **D .** None of the above